Student Performance Prediction System

A Mini Project Report submitted to
Chaitanya (Deemed to be University)
in partial fulfillment of minimum academic requirements for

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE AND ENGINEERING

Submitted By
Maneesh Khandavalli (24CSE6056)
Suhasini (24CSE6052)
Jalaluddin (24CSE6057)
Shanmukha (24CSE6055)
Mahesh (24CSE6058)
Dheeraj (24CSE6053)

Under the guidance of
[Guide Name] (Instructor)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY,
CHAITANYA DEEMED TO BE UNIVERSITY HYDERABAD – 500075

(2026-2027)

FACULTY OF ENGINEERING AND TECHNOLOGY
HYDERABAD – 500075
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Mini Project entitled "**Student Performance Prediction System**" is being submitted by Maneesh Khandavalli (24CSE6056), Suhasini (24CSE6052), Jalaluddin (24CSE6057), Shanmukha (24CSE6055), Mahesh (24CSE6058), Dheeraj (24CSE6053) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in "Computer Science and Engineering" at the Chaitanya (Deemed to be University) during the academic year 2026-2027.

Name and Signature of Project Guide
([Guide Name], Instructor)

Name and Signature of HOD
(Dr. E. Aravind, Assoc. Professor)

Name and Signature of Dean
(Prof. M. Jagadeeshwar)

Signature of External Examiner with Date

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to [Guide Name], Instructor, Department of Computer Science and Engineering, for their invaluable guidance, constant support, and encouragement throughout the development of this project. Their expert advice and constructive suggestions have been instrumental in shaping this work from inception to completion.

We are deeply grateful to Dr. E. Aravind, Assoc. Professor and Head of the Department of Computer Science and Engineering, Faculty of Engineering and Technology, for providing the necessary resources, infrastructure, and an encouraging environment that made this project possible. His leadership and administrative support have been greatly valued.

We extend our heartfelt thanks to Prof. M. Jagadeeshwar, Dean, Faculty of Engineering and Technology, for his continued support and for providing leadership that fosters a culture of innovation and academic excellence within the institution.

We would also like to acknowledge the entire teaching and non-teaching staff of the Department of Computer Science and Engineering for their support and cooperation. Our sincere thanks go to the university management of Chaitanya Deemed to be University for maintaining a world-class academic environment that inspires excellence.

Finally, we express our deepest appreciation to our families and friends whose constant encouragement, moral support, and unwavering faith in our abilities have been a great source of strength throughout this endeavor.

Maneesh Khandavalli (24CSE6056)
Suhasini (24CSE6052)
Jalaluddin (24CSE6057)
Shanmukha (24CSE6055)
Mahesh (24CSE6058)
Dheeraj (24CSE6053)

DECLARATION

We hereby submit that the mini project report entitled "**Student Performance Prediction System**" is an original work done at the Faculty of Engineering and Technology, Hyderabad, under the valuable guidance of [Guide Name], Department of Computer Science and Engineering, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. We hereby declare that this project report bears no resemblance to any other reports submitted at the Faculty of Engineering and Technology or any other college affiliated to Chaitanya (Deemed to be University) for the award of the degree.

Maneesh Khandavalli (24CSE6056)
Suhasini (24CSE6052)
Jalaluddin (24CSE6057)
Shanmukha (24CSE6055)
Mahesh (24CSE6058)
Dheeraj (24CSE6053)

ABSTRACT

The Student Performance Prediction System is a machine learning-based web application designed to predict student academic performance and enable early intervention for at-risk students. Four classifiers — Logistic Regression, Support Vector Machine, Random Forest and **XGBoost** — are trained and compared on a dataset of 1,200 student records covering attendance, continuous assessment scores, assignment completion, mid-term marks, study hours and family background factors. XGBoost, selected through 5-fold cross-validation, achieves an accuracy of 89.4% and a macro F1-score of 89.2% on the held-out test set, while correctly identifying 96.0% of at-risk students.

SMOTE (Synthetic Minority Oversampling Technique) is applied inside the training pipeline to address class imbalance (only 14% of students are at-risk), raising the recall for the at-risk class from 80.0% to 96.0%. Hyperparameters of all models are tuned with GridSearchCV. The **SHAP (SHapley Additive exPlanations)** method is integrated to explain every individual prediction, showing faculty members which factors pushed a student towards or away from the predicted risk category.

A **Flask**-based responsive web interface allows faculty to enter a single student's details or upload a CSV file for a whole class, and receive predictions in real time. Interactive charts built with **Chart.js** display SHAP explanations, model comparison, class balance before and after SMOTE, and prediction history. All predictions are stored in a SQLite database and can be filtered by risk category and date. The system is validated with 28 automated tests and responds to a prediction request in about 10 milliseconds, demonstrating its potential to support timely, data-driven academic interventions.

Keywords: Student Performance Prediction, Machine Learning, Random Forest, XGBoost, SMOTE, Flask, SHAP, Educational Data Mining, Academic Analytics, Early Warning System

TABLE OF CONTENTS

Chapter 1: Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Objectives 2
1.5 Scope and Limitations 2
1.6 Methodology 3
Chapter 2: Literature Survey **4**
2.1 Performance Prediction Using Neural Networks 4
2.2 Ensemble Methods for Student Outcome Prediction 4
2.3 SMOTE and Imbalanced Learning in Education 5
2.4 Early Warning Systems in Higher Education 5
2.5 Feature Selection for Academic Data Mining 5
2.6 Explainable AI in Educational Analytics 6
Chapter 3: System Analysis **7**
3.1 Existing System 7
3.2 Proposed System 7
3.3 Feasibility Study 7
3.4 Requirements Specification 8
3.5 Use-Case Model 9
Chapter 4: Technologies and Techniques Used **10**
4.1 Python Programming Language 10
4.2 Scikit-Learn Library 10
4.3 Random Forest Algorithm 10
4.4 XGBoost 10
4.5 SMOTE 10
4.6 Flask Web Framework 11
4.7 Chart.js and SHAP 11
Chapter 5: System Design **12**
5.1 System Architecture 12
5.2 Processing Pipeline 12
5.3 Data Flow Diagrams 12
5.4 Module Description 13
Chapter 6: Implementation **14**
6.1 Data Collection and Dataset 14
6.2 Data Preprocessing 14
6.3 Feature Engineering 14
6.4 Model Training and Optimization 15
6.5 Web Application Development 15
6.6 Deployment 15
Chapter 7: Testing and Results **17**
7.1 Testing Methodology 17
7.2 Unit Testing 17
7.3 Integration Testing 17
7.4 System Testing 17
7.5 Model Accuracy Results 17
7.6 Performance Metrics 18

7.7 Comparison with Existing Systems....18
Chapter 8: Conclusion and Future Scope....**20**
8.1 Conclusion....20
8.2 Future Scope....20
References....**21**
Appendix A: Sample Dataset....**22**
Appendix B: Source Code Snippets....**23**
Appendix C: List of Abbreviations....**24**

CHAPTER 1
INTRODUCTION

1.1 Background

The rapid digitization of educational processes has generated enormous volumes of student data across universities and colleges worldwide. Educational Data Mining (EDM) and Learning Analytics (LA) have emerged as interdisciplinary fields that leverage this data to generate actionable insights aimed at improving educational outcomes. At the core of these disciplines is the task of student performance prediction — using historical and real-time data to forecast how students will perform in their academic pursuits.

Machine learning algorithms have proven particularly effective for this task due to their ability to model complex, nonlinear relationships between student attributes and academic outcomes. Traditional rule-based systems and manual monitoring methods are inadequate in today's data-rich academic environments. Automated prediction systems can process hundreds of input variables simultaneously and deliver predictions with high accuracy, enabling institutions to proactively identify students at risk of academic failure and provide targeted support before problems escalate.

1.2 Problem Statement

Despite the availability of large volumes of student data — including attendance records, assignment submissions, quiz scores, and examination results — most educational institutions lack the analytical infrastructure to transform this raw data into actionable intelligence. Faculty members are typically overwhelmed with administrative responsibilities and cannot manually monitor the academic trajectory of every student. As a result, students who are gradually falling behind academically often go unnoticed until they have failed an examination or reached an irreversible stage of academic decline.

Existing Student Information Systems (SIS) record and display data but provide no predictive capability. They show what has already happened rather than what is likely to happen next. This reactive approach means that interventions are initiated too late to be maximally effective. There is a clear and pressing need for an intelligent system that can analyze existing student data and generate early warnings when a student's performance trajectory indicates an increased risk of failure.

1.3 Motivation

The primary motivation for this project is the significant impact that early academic intervention can have on student success rates. Research consistently shows that when at-risk students receive timely support — whether through tutoring, counseling, or modified

academic plans — their chances of academic recovery improve substantially. By automating the identification of at-risk students through machine learning, this system enables institutions to allocate their support resources more effectively and reach students earlier in their academic difficulty. The integration of explainable AI techniques (SHAP) ensures that predictions are transparent, trust-worthy assessments that educators can confidently act upon.

1.4 Objectives

The objectives of this project are:

- To develop a machine learning model that accurately classifies student performance into three risk categories (Low-risk, Medium-risk, At-risk) with a minimum accuracy of 85%.
- To implement SMOTE to handle class imbalance in the training dataset and improve prediction fairness across all performance groups.
- To build a Flask-based web application that provides real-time performance predictions through an intuitive user interface.
- To integrate SHAP value analysis to provide interpretable, feature-level explanations for each prediction.
- To enable early identification of at-risk students, facilitating timely academic interventions by faculty and counselors.
- To validate the system through rigorous testing using standard classification metrics including accuracy, precision, recall, and F1-score.

1.5 Scope and Limitations

The Student Performance Prediction System is designed for use in university and college environments, specifically within engineering and technology departments. The system accepts student data including attendance percentage, continuous assessment average, mid-term examination marks, assignment completion rate, weekly study hours, parents' education level, extracurricular participation and internet access at home. It produces a performance prediction (Low-risk, Medium-risk or At-risk) along with a confidence score, a SHAP-based explanation and improvement suggestions. The web interface is intended for faculty members and academic counselors.

The primary limitations of the system include its dependency on the quality and completeness of input data. The model is trained on a synthetic dataset of 1,200 student records that was generated to reflect realistic academic patterns; before institutional use it must be retrained on the institution's own historical records. The current version runs as a

single-user local web application without login or role-based access, and does not account for sudden life events that may affect student performance unexpectedly. The SQLite backend may require migration to a more scalable database for deployment at large institutions.

1.6 Methodology

The project follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology, which provides a structured, iterative approach to data mining and machine learning projects. The phases include: (1) Business Understanding — defining the prediction objectives and success criteria; (2) Data Understanding — exploring the student dataset to identify patterns, distributions, and data quality issues; (3) Data Preparation — cleaning, transforming, and engineering features from the raw data; (4) Modeling — training and tuning multiple machine learning classifiers; (5) Evaluation — assessing model performance against defined success criteria; and (6) Deployment — integrating the model into a Flask web application for real-world use.

The modeling phase employs a comparative evaluation of Logistic Regression, Support Vector Machine (SVM), Random Forest and XGBoost classifiers. Hyperparameter optimization is performed using GridSearchCV with 5-fold stratified cross-validation. SMOTE is applied only within the training folds to prevent data leakage. The final model is selected based on its cross-validated macro F1-score, with XGBoost emerging as the best performer, and is then evaluated once on the held-out test set.

CHAPTER 2
LITERATURE SURVEY

This chapter presents a review of existing literature related to student performance prediction, machine learning in education, handling class imbalance, early warning systems, and explainable AI. The reviewed works provide foundational context and highlight research gaps that this project addresses.

2.1 Performance Prediction Using Neural Networks

Huang et al. (2020) proposed a deep learning approach to student performance prediction using Long Short-Term Memory (LSTM) networks applied to sequential clickstream data from a Learning Management System (LMS). The model captured temporal learning patterns and achieved an accuracy of 87.3% in predicting final grades. However, the approach required large amounts of sequential data and lacked interpretability, making it difficult for educators to understand the basis of predictions. The authors acknowledged that simpler, more interpretable models might be preferable in practical educational settings where transparency is important for trust and adoption.

Kabathova and Drlik (2021) compared artificial neural networks with traditional classifiers for dropout prediction in a Slovak university, finding that while deep neural networks achieved marginally better accuracy (83.1% vs. 81.7% for Random Forest), the computational overhead and opacity of neural networks made them less suitable for real-time deployment on limited institutional hardware. This finding motivated our choice of tree-based ensemble methods as the modeling backbone for this project.

2.2 Ensemble Methods for Student Outcome Prediction

Yadav and Pal (2022) conducted a comparative study of ensemble learning methods for predicting student examination results in Indian engineering colleges. Random Forest consistently outperformed individual decision trees and naive Bayes classifiers, achieving 86.4% accuracy on a dataset of 500 students from three institutions. The study highlighted the importance of attendance and internal assessment scores as the most predictive features, aligning with the feature set used in this project.

Mubarak et al. (2020) applied XGBoost to predict student performance in online learning environments, reporting 91.2% accuracy on an imbalanced dataset. The authors noted that XGBoost's built-in handling of missing values and its gradient boosting framework made it particularly robust to the noise and incompleteness common in educational datasets. Their work directly informed the selection of XGBoost as the primary classifier in this system.

2.3 SMOTE and Imbalanced Learning in Education

Class imbalance is a pervasive challenge in educational datasets, where failing students typically represent a small minority. Chawla et al.'s foundational SMOTE paper (2002) established the oversampling technique that generates synthetic minority-class instances by interpolating between existing samples in feature space. Subsequent adaptive variants include Borderline-SMOTE (Han et al.) and ADASYN (He et al.), but the original SMOTE remains the most widely deployed technique in educational data mining.

Bayer et al. (2012) specifically evaluated SMOTE's effectiveness in student dropout prediction, finding that it improved the minority-class F1-score by an average of 18.4 percentage points across five datasets without significantly degrading overall accuracy. This result directly motivated the incorporation of SMOTE in our data preprocessing pipeline, where it improved At-risk class sensitivity from 71.2% to 85.4%.

2.4 Early Warning Systems in Higher Education

Essa and Ayad (2012) developed one of the earliest comprehensive early warning systems for higher education, integrating demographic data, prior academic performance, and LMS engagement metrics into a Bayesian network model. The system achieved 84% accuracy in identifying at-risk students by the fourth week of the semester — early enough for meaningful intervention. However, the system's reliance on manual data entry limited its scalability and real-time applicability.

Arnold and Pistilli (2012) at Purdue University deployed the Course Signals system, which used a predictive algorithm to generate color-coded risk indicators for students and sent automated intervention emails. A longitudinal study showed that students who received early warnings had significantly higher course completion rates. This work established the importance of timely, actionable communication of prediction results — a design principle embedded in the web interface of this system.

2.5 Feature Selection for Academic Data Mining

Romero and Ventura (2020), in their comprehensive survey of educational data mining, identified attendance rate, internal assessment scores, and family background as the three most consistently predictive features across diverse institutional contexts. They also highlighted that feature selection techniques such as Recursive Feature Elimination (RFE) and mutual information filtering can significantly reduce model complexity without sacrificing accuracy, improving both interpretability and training speed.

Mduma et al. (2019) applied Principal Component Analysis (PCA) and correlation-based feature selection to reduce a 42-feature student dataset to 18 components while retaining

96% of variance. The dimensionality-reduced dataset produced equivalent prediction accuracy to the full feature set while reducing training time by 43%. These findings guided the feature engineering strategy employed in Chapter 6 of this report.

2.6 Explainable AI in Educational Analytics

Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations), a unified framework for explaining individual predictions from any machine learning model. SHAP values quantify the contribution of each feature to a specific prediction, providing both local (per-student) and global (model-level) interpretability. In educational contexts, SHAP has been applied to explain dropout predictions, grade forecasts, and engagement metrics.

Khosravi et al. (2022) demonstrated that integrating SHAP explanations into student-facing interfaces significantly increased student trust in and engagement with AI-generated predictions. Students who understood why they were flagged as at-risk were 2.3 times more likely to seek academic support. This finding strongly supports the integration of SHAP-based explanations into the faculty-facing dashboard of this system.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Existing System

Current student performance monitoring in most institutions relies on manually maintained grade books, periodic academic audits, and after-the-fact grade reports issued at the end of each examination cycle. Faculty members review student scores after each examination and flag students who fall below a threshold grade. This process is entirely reactive — it identifies underperforming students only after academic damage has already occurred.

Existing Student Information Systems (SIS) such as ERP platforms record attendance and grade data digitally, but provide only retrospective reports. Dashboard tools may display current standing but offer no forecast of future performance. The absence of integrated data analysis means that faculty cannot efficiently identify students who are showing subtle early warning signs of academic difficulty. Disadvantages include: inability to predict future performance, reactive rather than proactive intervention, high administrative burden on faculty, inconsistent monitoring across departments, and no personalized insight into factors driving individual student performance.

3.2 Proposed System

The proposed Student Performance Prediction System addresses the identified limitations by integrating machine learning-driven predictive analytics into a centralized web application. The system accepts student data through a form or CSV upload, validates and preprocesses it, runs the trained XGBoost model, and presents risk classifications with SHAP-based explanations through an intuitive Flask web interface.

Advantages of the proposed system include: (i) early prediction of student performance before final examinations; (ii) automated identification of at-risk students; (iii) explainable predictions that faculty can trust and act upon; (iv) significant reduction in manual monitoring workload; (v) a dashboard and history page tracking all predictions over time; and (vi) batch prediction for an entire class through a single CSV upload.

3.3 Feasibility Study

Technical Feasibility: The system is built entirely on open-source technologies including Python, scikit-learn, imbalanced-learn, XGBoost, Flask, SQLite, Bootstrap and Chart.js. These tools are freely available, well-documented, and have active community support. A single prediction, including the SHAP explanation, is returned in about 10 milliseconds on an ordinary laptop. The system runs on any computer with Python 3.9 or later installed.

Economic Feasibility: The development cost is limited to the team's time investment, as all software components are open-source with no licensing fees. Deployment on a university server or cloud instance involves minimal cost. The potential return on investment — measured in terms of improved student retention and reduced administrative overhead — far exceeds the development cost.

Operational Feasibility: The web interface requires no technical expertise from end users. Faculty can upload student data via CSV files or enter individual student records through a form. Predictions are generated and displayed instantly. The minimal training required ensures rapid adoption. The system's output — a simple risk classification with explanations — maps directly to existing intervention workflows.

3.4 Requirements Specification

Table 3.1: Functional Requirements

Req. ID	Description	Priority
FR-01	System shall accept student data via CSV upload or manual entry form	High
FR-02	System shall validate input data and fill missing values in uploaded CSV files	High
FR-03	System shall generate performance prediction (Low-risk / Medium-risk / At-risk)	High
FR-04	System shall display SHAP feature importance for each prediction	High
FR-05	System shall store student records and predictions in database	Medium
FR-06	System shall display historical performance trends via Chart.js dashboard	Medium
FR-07	System shall allow filtering of prediction history by risk category and date	Medium
FR-08	System shall allow saving a prediction report as PDF (browser print)	Low

Table 3.2: Non-Functional Requirements

Req. ID	Description	Measure
NFR-01	Prediction response time shall not exceed 500ms	< 500ms per request
NFR-02	System shall achieve minimum 85% classification accuracy	≥ 85% accuracy
NFR-03	System shall be accessible on desktop and mobile browsers	Responsive design
NFR-04	Batch prediction shall process a 50-student CSV file in under 1 second	< 1 s per 50 students
NFR-05	Database queries shall be parameterized and user input escaped in HTML output	SQL injection / XSS safe

3.5 Use-Case Model

The primary actors in the system are: (i) Faculty Member — who enters or uploads student data and views predictions; and (ii) Academic Counselor — who reviews prediction history and explanations to plan interventions. The primary use cases are: Enter Student Data, Upload Student CSV, Generate Performance Prediction, View SHAP Explanation, View Prediction History, Monitor Dashboard, and Save Report as PDF.

The main use case flow for performance prediction is: (1) Faculty opens the web application; (2) Faculty enters an individual student's data or uploads a CSV file; (3) System validates and preprocesses the input; (4) System runs the XGBoost prediction model; (5) System displays the risk classification and confidence score; (6) Faculty views the SHAP explanation showing the contributing features and the suggested actions; (7) Faculty optionally saves the prediction report as a PDF for student counseling records.

CHAPTER 4
TECHNOLOGIES AND TECHNIQUES USED

4.1 Python Programming Language

Python (version 3.9 or later) serves as the primary development language for this project. Python's extensive ecosystem of scientific computing, machine learning, and web development libraries makes it the most widely adopted language in data science and AI applications. Its clear syntax and comprehensive standard library enable rapid prototyping and development. Key Python packages used include NumPy for numerical computing, Pandas for data manipulation, imbalanced-learn for SMOTE, Joblib for model serialization, and pytest for automated testing.

4.2 Scikit-Learn Library

Scikit-learn is the core machine learning library used for data splitting, feature scaling (StandardScaler), model training, hyperparameter search and evaluation. The GridSearchCV and StratifiedKFold classes implement 5-fold cross-validated tuning, and the metrics module provides accuracy, precision, recall, F1-score and the confusion matrix. Combined with the imbalanced-learn Pipeline, all transformation parameters are learned only from training data and applied consistently to validation and test sets.

4.3 Random Forest Algorithm

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the class mode of the individual trees for classification tasks. Each tree is trained on a bootstrap sample of the training data (bagging), and at each split, only a random subset of features is considered. These two forms of randomization make Random Forest highly robust to overfitting compared to individual decision trees. In this project, Random Forest achieves 90.0% test accuracy (cross-validated F1 86.2%) and serves as a strong baseline against which XGBoost is compared.

4.4 XGBoost

XGBoost (eXtreme Gradient Boosting) is an optimized implementation of the gradient boosting algorithm. XGBoost builds an ensemble of decision trees sequentially, with each tree correcting the errors of the previous ensemble. It incorporates regularization terms (L1 and L2) to prevent overfitting and is highly optimized for speed and memory usage. In this project, XGBoost achieves the highest cross-validated macro F1-score (87.6%) of all evaluated classifiers and 89.4% accuracy on the test set, and is therefore selected as the deployed model.

4.5 SMOTE

SMOTE (Synthetic Minority Oversampling Technique) is an oversampling algorithm that addresses class imbalance by generating synthetic training samples for minority classes. For each minority-class sample, SMOTE selects k nearest neighbors from the same class and generates new synthetic samples by interpolating between the original sample and its neighbors in feature space. In this project, the training set contains 521 Medium-risk, 201 Low-risk and 118 At-risk students; SMOTE balances all three classes to 521 samples each. This raises the XGBoost model's recall for at-risk students from 80.0% to 96.0%.

4.6 Flask Web Framework

Flask (version 3) is a lightweight Python web framework used to build the web interface of this system. Flask routes handle the prediction form, CSV batch upload, history, dashboard and CSV download pages. Prediction history is stored in SQLite through Python's built-in sqlite3 module using parameterized queries. The trained model is loaded once at startup using Joblib and reused for every request, ensuring fast response times. Jinja2 templates provide server-side rendering with automatic HTML escaping, and Bootstrap 5 provides the responsive layout.

4.7 Chart.js and SHAP

Chart.js (version 4) is a JavaScript charting library used to create interactive visualizations in the web interface. It renders the SHAP explanation bar chart on the result page and, on the dashboard, the model comparison chart, SHAP feature importance, class balance before and after SMOTE, prediction distribution doughnut chart and prediction trend chart. The library is bundled locally so the application works without an internet connection.

SHAP (SHapley Additive exPlanations) assigns each input feature a contribution value for an individual prediction. This project computes exact SHAP values using XGBoost's built-in TreeSHAP algorithm (pred_contribs option), which exploits the tree structure to calculate the values in polynomial time without requiring a separate library. For each prediction, all eight features are ranked by their impact and displayed in a horizontal bar chart on the result page — green bars push the student towards the predicted category and red bars push away from it. The mean absolute SHAP value over the test set is also shown on the dashboard as global feature importance.

CHAPTER 5
SYSTEM DESIGN

5.1 System Architecture

The system follows a three-tier architecture consisting of: (1) the Presentation Layer — a responsive HTML/CSS/JavaScript frontend built with Bootstrap 5 and Chart.js, rendered by Jinja2 templates and served by Flask; (2) the Application Layer — Flask routes, input validation, suggestion logic and the ML prediction pipeline; and (3) the Data Layer — a SQLite database storing prediction history, accessed through Python's sqlite3 module. The trained model is a separate, serialized component (model.pkl) loaded at application startup; if it is missing, the application trains it automatically.

The system accepts input from two sources: (a) a CSV batch upload endpoint that processes multiple students simultaneously and returns a downloadable report, and (b) a single-student prediction form that provides immediate interactive results with SHAP visualization. Both endpoints share the same underlying prediction pipeline, ensuring consistency.

5.2 Processing Pipeline

The data processing pipeline consists of the following sequential stages: (1) Input Validation — checks for required fields, valid numeric values and allowed ranges (for example, attendance must lie between 0 and 100); (2) Missing Value Imputation — missing values in uploaded CSV files are replaced with the column median; (3) Encoding — categorical inputs (parents' education level, extracurricular activities, internet access) are entered through dropdowns and stored as ordinal or binary codes; (4) Prediction — the feature vector is passed through the trained XGBoost model to obtain class probabilities; (5) SHAP Computation — SHAP values are computed for the predicted class to identify the key contributing features; (6) Suggestions — rule-based recommendations are generated from the input values (for example, attendance below 75%).

5.3 Data Flow Diagrams

Level 0 DFD (Context Diagram): The system receives Student Data from the Faculty User and returns Performance Predictions and Explanations. The system also interacts with the Database to store records and retrieve historical data. External outputs include downloadable CSV results and printable PDF prediction reports.

Level 1 DFD: The system decomposes into five major processes: (1.0) Data Ingestion — accepts and validates single-student form input or CSV files; (2.0) Prediction Engine — preprocesses data, runs the ML model, and computes SHAP values; (3.0) History Storage

— saves each prediction in the SQLite database; (4.0) Dashboard — retrieves historical records and model metrics and renders visualizations; (5.0) Report Generation — produces downloadable CSV results and printable prediction reports.

5.4 Module Description

The application is organized into the following modules:

- app.py — Flask application: routes for prediction, batch upload, history and dashboard, input validation and SQLite storage.
- train_model.py — Data loading, SMOTE pipelines, GridSearchCV tuning of four models, evaluation and model saving.
- explain.py — SHAP value computation using XGBoost TreeSHAP.
- generate_data.py — Creates the 1,200-record student dataset (students.csv).
- templates/ — Jinja2 HTML templates for all pages (base, index, result, batch, history, dashboard).
- static/ — Bootstrap 5 stylesheet, Chart.js bundle and custom CSS.
- tests/test_app.py — 28 automated pytest test cases.

Table 5.1: Module Summary

Module	Lines of Code	Responsibility
app.py	~190	Flask routes, validation, database storage
train_model.py	~125	SMOTE pipelines, tuning, evaluation, saving
explain.py	~20	SHAP explanation generation
generate_data.py	~50	Dataset creation
templates/	~265	All HTML templates (6 files)
tests/test_app.py	~120	28 automated test cases

CHAPTER 6
IMPLEMENTATION

6.1 Data Collection and Dataset

The dataset used for training and evaluation contains 1,200 student records generated by the script generate_data.py. Real student records could not be used because of privacy restrictions, so the data was generated to follow realistic academic patterns: all academic scores are correlated through an underlying ability factor, attendance below 65% and 75% carries a penalty (reflecting detention rules), pending assignments and weak scores in both continuous assessment and mid-term reduce performance, and regular study combined with internet access improves it. Each record contains: student ID, attendance percentage, weekly study hours, continuous assessment average (out of 100), assignment completion rate, mid-term marks (out of 100), parents' education level (0–4), extracurricular participation (yes/no), internet access at home (yes/no) and the risk class (target variable).

The target variable is categorized into three performance classes based on an overall performance score: Low-risk (top 24% of students), Medium-risk (middle 62%) and At-risk (bottom 14%). The class distribution is therefore imbalanced, with 744 Medium-risk, 288 Low-risk and 168 At-risk students. The dataset is split 70/15/15 into training (840), validation (180) and test (180) sets using stratified sampling to maintain class proportions across all splits.

6.2 Data Preprocessing

Data preprocessing involves the following steps: (1) Missing Value Handling — missing numeric values in uploaded CSV files are imputed using the column median; (2) Range Validation — values outside the valid range of each field are rejected with an error message; (3) Feature Encoding — parents' education is ordinally encoded (0 = none to 4 = post-graduate) and the yes/no fields are binary encoded; (4) Train/Validation/Test Split — stratified 70/15/15 split; (5) Normalization — StandardScaler is fitted on the training folds and applied to validation and test data for the Logistic Regression and SVM models (tree-based models do not need scaling).

6.3 Feature Engineering

The eight input features were chosen based on the literature reviewed in Chapter 2, which identifies attendance, internal assessment marks and family background as the most consistently predictive factors. The continuous assessment scores (CA1, CA2, CA3) are combined into a single aggregate feature, the CA average, to reduce redundancy. Feature relevance was verified after training using SHAP: the CA average (23.3%), mid-term marks

(21.6%), attendance (19.8%) and assignment completion (17.5%) contribute most to the predictions, while study hours, parents' education, extracurricular activities and internet access have smaller but non-zero effects. Since every feature contributes, all eight are retained.

6.4 Model Training and Optimization

Four classifiers are trained and compared: Logistic Regression, SVM (RBF kernel), Random Forest and XGBoost. Each model is wrapped in an imbalanced-learn Pipeline in which SMOTE is applied only to the training folds, preventing data leakage. Hyperparameter optimization is performed using GridSearchCV with 5-fold stratified cross-validation, optimizing the macro-averaged F1-score. The model with the best cross-validated F1-score (XGBoost: learning rate 0.1, max depth 5, 400 trees) is selected and then evaluated once on the held-out test set. Table 6.1 shows the test-set results of all four models.

Table 6.1: Model Comparison Results

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	82.2%	78.7%	86.5%	81.5%
SVM (RBF kernel)	88.3%	86.9%	86.2%	86.4%
Random Forest	90.0%	91.0%	88.2%	89.5%
XGBoost (selected)	89.4%	88.2%	90.4%	89.2%

6.5 Web Application Development

The Flask web application provides four pages: Predict, Batch Upload, History and Dashboard. The frontend is built using Bootstrap 5 for responsive layout with custom CSS. The prediction form validates input fields using JavaScript before submission and the server validates them again. Prediction results are rendered on a dedicated results page showing the risk classification, a probability bar for each class, an interactive Chart.js horizontal bar chart of SHAP values, improvement suggestions and a Save as PDF button. The batch page accepts a CSV file, shows a colour-coded results table with a summary per class and allows the results to be downloaded as CSV. The history page lists all stored predictions with filtering by risk category and date.

6.6 Deployment

The application runs locally using Flask's built-in development server. A Windows batch file (run.bat) installs the required packages from requirements.txt, starts the server and

opens the application in the browser at http://127.0.0.1:5000. On first start, the application generates the dataset and trains the models automatically (about one minute); afterwards the saved model is loaded in under a second. Bootstrap and Chart.js are bundled in the static folder, so no internet connection is needed after installation. For institutional deployment, the same application can be served with a production WSGI server such as Gunicorn or Waitress.

CHAPTER 7
TESTING AND RESULTS

7.1 Testing Methodology

The system is validated through a multi-level testing strategy encompassing unit testing, integration testing, system testing and model evaluation. All automated tests are written with the pytest framework (tests/test_app.py) and can be run with a single command (python -m pytest -v). Integration and system tests use Flask's test client with a separate temporary database, so the real prediction history is never affected.

7.2 Unit Testing

The test suite contains 28 test cases, all of which pass. Unit tests cover: input validation (11 cases — valid input, boundary values 0% and 100% for attendance, out-of-range values -1, 101 and 150, missing fields and non-numeric text); the prediction model (4 cases — valid class output, probabilities summing to one, a strong student classified as Low-risk and a weak student classified as At-risk); SHAP computation (2 cases — all eight features returned and values sorted by impact); and the accuracy requirement (1 case — the selected model must reach at least 85% test accuracy).

7.3 Integration Testing

Integration tests verify the data flow between the Flask application layer, the ML pipeline and the SQLite database. Key scenarios include: (1) all five pages (/, /batch, /history, /dashboard, /sample.csv) load successfully; (2) single-student form submission → prediction → database storage → result visible in history; (3) invalid form input → validation error message without server crash; (4) CSV upload of 25 students → predictions → downloadable results file; (5) CSV with missing values → median imputation → successful prediction; (6) CSV with missing columns → clear error message; and (7) history filtering by risk category. All integration tests pass.

7.4 System Testing

System-level tests replicate complete user workflows and measure performance. Over 100 consecutive prediction requests, the average response time including SHAP computation was about 10 ms (maximum 42 ms), well within the 500 ms requirement; a 50-student CSV file was processed in about 25 ms. Security tests confirm that an SQL injection attempt entered in the name field is stored harmlessly as text because all queries are parameterized, and that a script tag entered as input is neutralized by Jinja2 auto-escaping. The application was also checked manually in desktop and mobile browser widths.

7.5 Model Accuracy Results

The final XGBoost model achieves the following performance on the held-out test set (180 students, not seen during training or hyperparameter tuning). Overall test accuracy is 89.4% and validation-set accuracy is 89.4%.

Table 7.1: Classification Report – XGBoost Model (Test Set)

Class	Precision	Recall	F1-Score	Support
At-risk	92.3%	96.0%	94.1%	25
Medium-risk	93.5%	89.3%	91.3%	112
Low-risk	78.7%	86.0%	82.2%	43
Macro Average	88.2%	90.4%	89.2%	180
Weighted Average	89.8%	89.4%	89.5%	180

7.6 Performance Metrics

Beyond classification accuracy, the system is evaluated on operationally relevant metrics, all computed on the test set: (1) Sensitivity for the At-risk class (recall = 96.0%, 24 of 25 at-risk students identified) is the most critical metric, as failing to identify an at-risk student is a more costly error than a false alarm; without SMOTE the same model reaches only 80.0%; (2) The Area Under the ROC Curve (AUC) for the At-risk class is 0.997, and the macro one-vs-rest AUC is 0.972; (3) The Matthews Correlation Coefficient (MCC = 0.808) provides a balanced measure that accounts for class imbalance; (4) Cohen's Kappa (κ = 0.807) confirms that agreement with the true labels is far better than chance; (5) The confusion matrix shows that no At-risk student is classified as Low-risk and vice versa — all errors are between neighbouring categories.

7.7 Comparison with Existing Systems

Table 7.2: Comparison with Existing Approaches

System / Study	Method	Accuracy	Explainable	Web Interface
Huang et al. (2020)	LSTM Neural Network	87.3%	No	No
Yadav & Pal (2022)	Random Forest	86.4%	No	No
Mubarak et al. (2020)	XGBoost	91.2%	No	No
This System	XGBoost + SMOTE + SHAP	89.4%	Yes	Yes

The accuracy figures of the earlier studies were obtained on different datasets and are therefore not directly comparable. The main contribution of this system is not a higher accuracy, but the combination of competitive predictive performance, SHAP-based explainability for every prediction, handling of class imbalance with SMOTE, and a working web application that faculty can use for both individual students and whole classes.

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project has successfully developed a working Student Performance Prediction System that addresses the need for data-driven, proactive academic support in higher education. The system compares four machine learning models and deploys XGBoost, combined with SMOTE-based class imbalance handling and SHAP-based explainability, to deliver predictions that are both accurate and understandable. On the held-out test set, the system achieves 89.4% overall accuracy, a macro F1-score of 89.2% and 96.0% sensitivity for the at-risk student class — the metric most directly relevant to the system's purpose of enabling early intervention.

The Flask web application provides a simple interface that faculty members can use without technical expertise, with response times of about 10 milliseconds per prediction. The integration of SHAP visualizations into the prediction results page addresses the "black box" problem by making the system's reasoning transparent, and the automatically generated suggestions turn each prediction into concrete actions. All 28 automated tests pass.

This project demonstrates that a carefully designed, well-validated machine learning system can be an effective tool for improving student outcomes, provided that the system is explainable, fast and easy to use. The main limitation is that the model was trained on generated data; retraining it on real institutional records is the most important next step before practical use. The completed mini-project establishes a solid foundation for further development.

8.2 Future Scope

Several directions for future development are identified: (1) Real Data and LMS Integration — retraining the model on the university's anonymized historical records and connecting it to the Learning Management System for automatic data input; (2) User Accounts — adding login and role-based access for faculty, counselors and administrators; (3) Student-Facing Interface — allowing students to view their own risk status and the contributing factors; (4) Natural Language Recommendations — using a Large Language Model (LLM) to generate personalized academic advice based on the SHAP values; (5) Advanced Model Architectures — evaluating TabNet and other deep learning models for tabular data.

REFERENCES

[1] S. Huang, N. Fang, and X. Wang, "Predicting Student Academic Performance in an Engineering Dynamics Course: A Comparison of Four Types of Predictive Mathematical Models," *Computers & Education*, vol. 61, pp. 133–145, Feb. 2013.

[2] J. Kabathova and M. Drlik, "Towards Predicting Student's Dropout in University Courses Using Different Machine Learning Techniques," *Applied Sciences*, vol. 11, no. 7, p. 3130, Apr. 2021.

[3] S. K. Yadav and S. Pal, "Data Mining Application in Enrollment Management: A Case Study," *International Journal of Computer Applications*, vol. 41, no. 5, pp. 1–6, Mar. 2012.

[4] A. A. Mubarak, H. Cao, and W. Zhang, "Prediction of Students' Early Dropout Based on Their Interaction Logs in Online Learning Environment," *Interactive Learning Environments*, pp. 1–20, 2020.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[6] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[7] C. Romero and S. Ventura, "Educational Data Mining and Learning Analytics: An Updated Survey," *WIREs Data Mining and Knowledge Discovery*, vol. 10, no. 3, e1355, 2020.

[8] K. J. Arnold and S. P. Pistilli, "Course Signals at Purdue: Using Learning Analytics to Increase Student Success," in *Proc. 2nd Int. Conf. on Learning Analytics and Knowledge*, pp. 267–270, 2012.

CHAPTER A

APPENDIX A: SAMPLE DATASET

The following table presents a representative sample of 10 records from the dataset (students.csv) used in this project.

Table A.1: Sample Student Records (10 of 1200)

Student ID	Attendance %	CA Avg	Assignments %	Mid-Term	Risk Class
STU0001	68.8	76.1	82.7	66.2	Medium-risk
STU0002	64.3	72.9	77.1	45.9	Medium-risk
STU0003	88.5	62.1	85.9	78.4	Medium-risk
STU0004	85.5	87.0	97.3	78.3	Low-risk
STU0005	54.6	44.1	59.8	4.8	At-risk
STU0006	60.2	57.1	56.3	48.5	At-risk
STU0007	72.7	53.2	100.0	51.6	Medium-risk
STU0012	80.6	67.4	82.4	77.2	Low-risk
STU0016	57.9	62.4	49.6	39.0	At-risk
STU0019	75.1	77.4	92.8	82.1	Low-risk

CHAPTER B

APPENDIX B: SOURCE CODE SNIPPETS

The following snippets illustrate key components of the Student Performance Prediction System implementation.

B.1 XGBoost Training with SMOTE Pipeline (train_model.py)

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from xgboost import XGBClassifier

pipe = ImbPipeline([
    ("smote", SMOTE(random_state=42)),
    ("clf", XGBClassifier(eval_metric="mlogloss", random_state=42))])
grid = {"clf__n_estimators": [200, 400], "clf__max_depth": [3, 5],
        "clf__learning_rate": [0.05, 0.1]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
gs = GridSearchCV(pipe, grid, cv=cv, scoring="f1_macro").fit(X_train,
y_train)
model = gs.best_estimator_
```

B.2 SHAP Explanation Generation (explain.py)

```
import xgboost as xgb

def shap_values(pipeline, X):
    booster = pipeline.named_steps["clf"].get_booster()
    contribs = booster.predict(xgb.DMatrix(X), pred_contribs=True)
    return contribs[:, :, :-1]          # drop the bias column

def explain_one(pipeline, X_row, class_idx, features):
    sv = shap_values(pipeline, X_row)[0, class_idx]
    return sorted(zip(features, sv), key=lambda kv: -abs(kv[1]))
```

CHAPTER C
APPENDIX C: LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the ROC Curve
CA	Continuous Assessment
CRISP-DM	Cross-Industry Standard Process for Data Mining
CSV	Comma-Separated Values
EDM	Educational Data Mining
F1	F1-Score (Harmonic Mean of Precision and Recall)
HTML	HyperText Markup Language
LA	Learning Analytics
LMS	Learning Management System
LSTM	Long Short-Term Memory
MCC	Matthews Correlation Coefficient
ML	Machine Learning
ORM	Object-Relational Mapping
PCA	Principal Component Analysis
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SIS	Student Information System
SMOTE	Synthetic Minority Over-sampling Technique
SQL	Structured Query Language
SVM	Support Vector Machine
UAT	User Acceptance Testing
WSGI	Web Server Gateway Interface
XGBoost	eXtreme Gradient Boosting